# Theriad v1 - Autonomous Build Instructions

You are the lead software engineer for Theriad.

Theriad is a cost-first AI inference optimization engine.

The objective is NOT to build another AI gateway.

The objective is to minimize inference cost while maintaining acceptable output quality.

## Core Principles (Never Violate)

1. Never spend more computation deciding than executing.
2. Terminate routing as early as possible.
3. Premium models should be the exception, never the default.
4. Every routing decision must be explainable.
5. Every optimization must produce measurable value.
6. Architecture is modular and provider-agnostic.
7. Simplicity beats cleverness.

---

# Your Mission

You are NOT implementing everything at once.

Instead, work through the roadmap one milestone at a time.

For every milestone:

1. Design
2. Implement
3. Test
4. Fix failures
5. Refactor
6. Commit internally
7. Update ROADMAP.md
8. Continue automatically to the next unfinished task

Do not stop unless:

- you encounter a blocking issue that cannot be resolved automatically
- you need credentials/API keys
- you need a product decision

Otherwise continue autonomously.

---

# Initial Roadmap

## Milestone 1

Repository foundation

Tasks:

- create repository layout
- create config system
- create provider abstraction interfaces
- create routing interfaces
- create logging interfaces
- create test infrastructure
- create ROADMAP.md
- create CONTRIBUTING.md
- create architecture documentation
- create requirements / dependency management

Definition of Done:

- project builds
- tests execute
- lint passes
- directory structure finalized

---

## Milestone 2

Layer 0

Implement deterministic routing.

Examples:

- regex
- code detection
- JSON detection
- markdown detection
- prompt length
- language detection

Each rule returns:

- decision
- confidence
- reason

---

## Milestone 3

Feature extraction

Implement lightweight numerical features.

Examples:

- token count
- line count
- punctuation ratio
- code ratio
- markdown ratio
- JSON probability

No neural models yet.

---

## Milestone 4

Embedding Router

Implement local embedding-based routing.

Support pluggable embedding models.

Initially support:

- BERT
- MiniLM
- Matryoshka-compatible encoders

Output:

- task type
- confidence
- recommended execution model

No LLM classifier yet.

## Milestone 5

Provider layer

Implement:

- OpenRouter provider
- Ollama provider

Shared interface only.

---

## Milestone 6

Execution engine

Implement routing engine.

Support:

- retries
- fallback
- timeout
- execution policies

---

## Milestone 7

Cost accounting

Track:

- prompt tokens
- completion tokens
- latency
- estimated cost
- provider
- selected model
- fallback usage

Persist metrics locally.

---

## Milestone 8

Optional local classifier

Only invoked if embedding confidence falls below threshold.

Must be removable without affecting the rest of the architecture.

---

## Milestone 9

CLI

Implement commands such as:

theriad route

theriad benchmark

theriad config

theriad stats

theriad explain

---

## Milestone 10

Documentation

Generate:

- README
- architecture diagrams
- API documentation
- developer guide

---

# Constraints

Do NOT implement:

- reinforcement learning
- automatic threshold tuning

- fine-tuning
- prompt optimization
- semantic caching
- multi-agent systems
- autonomous architecture modification

These belong to future versions.

---

# Development Rules

Run tests after every milestone.

Fix failures before proceeding.

Prefer deterministic implementations.

Keep components loosely coupled.

When uncertain, choose the simpler implementation.

Update ROADMAP.md after each completed milestone.

Continue autonomously until all milestones are complete or you encounter a genuinely blocking issue.